Московский авиационный институт
(национальный исследовательский институт)

Институт «Компьютерные науки и прикладная математика»

Курсовой проект
«Теория автоматов и формальных языков»
VII семестр

Студент: Колесник Д.С.
Группа: М8О-413Б-23
Руководитель: к.т.н, доцент ОЦ 8 Киндинова В. В.

Оценка:
Дата:

Москва - 2026

1. Проектирование регулярной леволинейной грамматики

Заданный паттерн ^\d{5}$ описывает регулярный язык L над терминальным алфавитом V_T = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9}, порождающий цепочки, состоящие ровно из пяти десятичных цифр (ZIP-код).

По требованию задания проектируется леволинейная грамматика $G_L = (V_T, V_N, P, S)$. В леволинейной грамматике все правила вывода имеют канонический вид:

A -> Ba или A -> a, где A, B принадлежат V_N, а принадлежит V_T.

В леволинейной грамматике цепочка в процессе вывода нарастает слева направо (нетерминал порождает префикс слова, к которому справа приписывается очередной терминал).

Определим формальные компоненты леволинейной грамматики G_L:

* V_T = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9} — терминальный алфавит;
* V_N = {S, A, B, C, D} — множество нетерминальных символов;
* S — целевой (начальный) нетерминал грамматики.

Множество правил вывода (продукций) P:

Нетерминал	Правила вывода (леволинейные)
A	A -> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
B	B -> A0 | A1 | A2 | A3 | A4 | A5 | A6 | A7 | A8 | A9
C	C -> B0 | B1 | B2 | B3 | B4 | B5 | B6 | B7 | B8 | B9
D	D -> C0 | C1 | C2 | C3 | C4 | C5 | C6 | C7 | C8 | C9
S	S -> D0 | D1 | D2 | D3 | D4 | D5 | D6 | D7 | D8 | D9

Здесь запись вида «A -> 0 | 1 | ... | 9» и «B -> A0 | A1 | ... | A9» является компактной формой записи для десяти отдельных продукций: B -> A0, B -> A1, ..., B -> A9.

Семантика нетерминалов (накопление префикса распознанной длины):

* A — порождает цепочку из ровно 1 цифры (начальный этап вывода);
* B — накапливает цепочку из ровно 2 цифр (B -> Ad);
* C — накапливает цепочку из ровно 3 цифр (C -> Bd);
* D — накапливает цепочку из ровно 4 цифр (D -> Cd);
* S — целевой нетерминал: цепочка из ровно 5 цифр (S -> Dd).

Пример цепочки вывода для слова «12345» (вывод идёт от целевого символа S):

S => D5 => C45 => B345 => A2345 => 12345

Поскольку все правила имеют вид A -> Ba или A -> a, грамматика G_L является леволинейной регулярной (тип 3 по классификации Хомского).

2. Преобразование леволинейной грамматики в конечный автомат (КА). Анализ ДКА / НДКА

По каноническому алгоритму преобразования леволинейной грамматики $G_L = (V_T, V_N, P, S)$ в конечный автомат $M = (Q, \Sigma, \delta, q_0, F)$:

1. Вводится новое начальное состояние q_0 (которое не принадлежит V_N): начальное состояние автомата q_0.

2. Каждому нетерминалу из V_N сопоставляется состояние автомата:

A -> q_1, B -> q_2, C -> q_3, D -> q_4, S -> q_f

3. Для каждого правила вида A -> a (a принадлежит V_T) задаётся переход из начального состояния: $\delta(q_0, a) = A$ (то есть $\delta(q_0, d) = q_1$).

4. Для каждого правила вида Y -> Xa (X, Y принадлежат V_N, a принадлежит V_T) задаётся переход: $\delta(X, a) = Y$:

$\delta(A, d) = B \Rightarrow \delta(q_1, d) = q_2$
$\delta(B, d) = C \Rightarrow \delta(q_2, d) = q_3$
$\delta(C, d) = D \Rightarrow \delta(q_3, d) = q_4$
$\delta(D, d) = S \Rightarrow \delta(q_4, d) = q_f$

5. Целевой нетерминал S становится допускающим состоянием автомата: $F = \{S\} = \{q_f\}$.

6. Для обеспечения полной определённости функции переходов вводится состояние-ловушка q_{err}: $\delta(q_f, d) = q_{err}$, $\delta(q_{err}, d) = q_{err}$.

Характеристики полученного автомата $M = (Q, \Sigma, \delta, q_0, F)$:

* $Q = \{q_0, q_1, q_2, q_3, q_4, q_f, q_{err}\}$ — множество состояний;
* $\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ — входной алфавит (далее обозначен символом d);
* q_0 — начальное состояние;
* $F = \{q_f\}$ — множество допускающих состояний.

Таблица переходов конечного автомата:

Состояние	Символ d (0-9)
-> q_0	q_1
q_1	q_2
q_2	q_3
q_3	q_4
q_4	q_f *
* q_f	q_{err}
q_{err}	q_{err}

Проверка на детерминированность:

Автомат называется детерминированным (ДКА), если:

1. В нём отсутствуют ε-переходы;

2. Для любого состояния q принадлежит Q и любого входного символа d принадлежит Σ функция δ(q, d) однозначно определяет не более одного следующего состояния.

В полученном автомате для каждого состояния и каждого символа d существует ровно один переход. Следовательно, автомат уже является ДКА. Выполнение алгоритма детерминизации (НДКА -> ДКА методом подмножеств) не требуется.

Диаграмма переходов ДКА:

-> (q_0) --[d]--> (q_1) --[d]--> (q_2) --[d]--> (q_3) --[d]--> (q_4) --[d]--> ((q_f)) --[d]--> (q_{err}) --[d]--> (q_{err})

Обозначения: -> — вход в начальное состояние; (( )) — допускающее состояние; [d] — переход по любой цифре d принадлежит {0..9}.

Примеры трассировки работы ДКА:

1) Вход «12345» (ровно 5 цифр):

q_0 --[1]--> q_1 --[2]--> q_2 --[3]--> q_3 --[4]--> q_4 --[5]--> q_f [Цепочка допущена: q_f принадлежит F]

2) Вход «1234» (4 цифры — неполная цепочка):

q_0 --[1]--> q_1 --[2]--> q_2 --[3]--> q_3 --[4]--> q_4 [Цепочка отклонена: q_4 не принадлежит F]

3) Вход «123456» (6 цифр — избыточная длина):

q_0 --[1]--> q_1 --[2]--> q_2 --[3]--> q_3 --[4]--> q_4 --[5]--> q_f --[6]--> q_{err} [Цепочка отклонена: q_{err} не принадлежит F]

3. Минимизация ДКА

Процесс минимизации ДКА состоит из двух последовательных этапов: удаление недостижимых состояний и слияние эквивалентных (неразличимых) состояний.

3.1. Алгоритм устранения недостижимых состояний

Состояние q принадлежит Q называется достижимым, если существует хотя бы одно входное слово w принадлежит Σ*, такое что $\delta^*(q_0, w) = q$. В противном случае состояние является недостижимым.

Выполним алгоритм поиска достижимых состояний (BFS) из начального состояния q_0:

* Шаг 0: $U_0 = \{q_0\}$
* Шаг 1: $U_1 = U_0 \cup \delta(q_0, d) = \{q_0, q_1\}$
* Шаг 2: $U_2 = U_1 \cup \delta(q_1, d) = \{q_0, q_1, q_2\}$
* Шаг 3: $U_3 = U_2 \cup \delta(q_2, d) = \{q_0, q_1, q_2, q_3\}$
* Шаг 4: $U_4 = U_3 \cup \delta(q_3, d) = \{q_0, q_1, q_2, q_3, q_4\}$
* Шаг 5: $U_5 = U_4 \cup \delta(q_4, d) = \{q_0, q_1, q_2, q_3, q_4, q_f\}$
* Шаг 6: $U_6 = U_5 \cup \delta(q_f, d) = \{q_0, q_1, q_2, q_3, q_4, q_f, q_{err}\}$
* Шаг 7: $U_7 = U_6 \cup \delta(q_{err}, d) = U_6$ (стабилизация).

Результат: множество достижимых состояний равно всему Q (все 7 состояний достижимы). Недостижимых состояний нет.

3.2. Алгоритм минимизации методом таблицы различимости

Два состояния p и q называются различимыми, если существует цепочка w принадлежит Σ*, которая переводит одно из них в допускающее состояние, а другое — в недопускающее.

Шаг 1 (базовое разбиение). Любая пара состояний (p, q), где одно состояние принадлежит F, а второе не принадлежит F, очевидно различима пустой цепочкой ε (|ε| = 0).

Поскольку $F = \{q_f\}$, а множество нефинальных состояний $Q \setminus F = \{q_0, q_1, q_2, q_3, q_4, q_{err}\}$, все пары (q_f, q_i) для любого $q_i \neq q_f$ помечаются как различимые (символ X).

Шаг 2 (итерационное распространение различимости). Для каждой пары (p, q) проверяем: если для входного символа d пара $(\delta(p, d), \delta(q, d))$ уже помечена как различимая, то и пара (p, q) помечается как различимая.

Проверка пар:

* Пара (q_4, q_{err}): $\delta(q_4, d) = q_f$, $\delta(q_{err}, d) = q_{err}$. Пара (q_f, q_{err}) помечена на шаге 1 -> пара (q_4, q_{err}) различима (X).
* Пара (q_3, q_4): $\delta(q_3, d) = q_4$, $\delta(q_4, d) = q_f$. Пара (q_4, q_f) помечена на шаге 1 -> пара (q_3, q_4) различима (X).
* Аналогично по индукции цепочки переходов различают все пары (q_i, q_j) при $i \neq j$, так как состояния q_i и q_j требуют разного количества цифр до попадания в q_f.

Таблица различимости состояний (X — состояния различимы, пустота — диагональ):

	q_0	q_1	q_2	q_3	q_4	q_f
q_1	X					
q_2	X	X				
q_3	X	X	X			
q_4	X	X	X	X		
q_f	X	X	X	X	X	
q_{err}	X	X	X	X	X	X

Вывод по минимизации:

Все пары различных состояний автомата оказались различимыми. Следовательно, ни одна пара состояний не может быть эквивалентна или объединена. Построенный ДКА уже является минимальным.

Минимальный автомат содержит 7 состояний: $q_0, q_1, q_2, q_3, q_4, q_f, q_{err}$.

Этот результат строго согласуется с фундаментальной теоремой теории автоматов: для языка слов фиксированной длины n над алфавитом Σ размерность минимального ДКА составляет в точности n + 2 состояния (n состояний счётчика длины, 1 допускающее состояние и 1 состояние-ловушка для цепочек длины > n).